



**HOCHSCHULE
MITTWEIDA**
University of Applied Sciences

BACHELOR THESIS

Mr.
Georgii Burdin

**Detecting Model Drift in
LLM-as-a-Service APIs: A Benchmarking
Framework for Campaign-Relevance
Classification**

Mittweida, April 2026



Faculty of **Applied Computer Sciences and Biosciences**

BACHELOR THESIS

Detecting Model Drift in LLM-as-a-Service APIs: A Benchmarking Framework for Campaign-Relevance Classification

Author:

Georgii Burdin

Course of Study:

B.Sc. Applied Mathematics

Seminar Group:

MA22w1-B

First Examiner:

Prof <Name> <First Examiner>

Second Examiner:

<Second Examiner>

Submission:

Mittweida, 01.04.2026

Defense/Evaluation:

Mittweida, 2026

Bibliographic Description

Burdin, Georgii:

Detecting Model Drift in LLM-as-a-Service APIs: A Benchmarking Framework for Campaign-Relevance Classification. – 2026. – 15 S.

Mittweida, Hochschule Mittweida – University of Applied Sciences, Faculty of Applied Computer Sciences and Biosciences, Bachelor Thesis, 2026.

Referat

1. Context

Gocomo uses API-based large language models (LLMs) for campaign-relevance classification on social-media posts.

2. Practical Problem

These API models change over time without full transparency, so the performance of a deployed classification pipeline may degrade.

3. Operational Constraint

Re-running a full benchmark frequently is too expensive.

4. Methodological Problem

Therefore, a framework is needed that can:

- represent the task consistently,
- monitor performance over time with a smaller but informative sample,
- detect drift statistically,
- support cost-aware model selection,
- and possibly recover performance through prompt optimisation.

Contents

Contents	I
1 Introduction	1
1.1 Context and Motivation	1
1.2 Problem Statement	1
1.3 Research Questions	2
2 Data and Evaluation Setup	3
2.1 Formal Problem Definition	3
2.1.1 Example LLM Request Payload	3
2.1.2 The Probability Space	4
2.1.3 LLM as a Parameterized Conditional Distribution	4
2.1.4 Loss Function and Risk	4
2.2 The 1200-Example Reference Benchmark	5
2.2.1 Sample Size Determination via Annotation Noise Floor	5
2.3 The 300-Example Monitoring Subset	6
2.4 Statistical Justification of the Monitoring Subset	6
2.4.1 Informative Sampling via Model Disagreement	6
2.4.2 Sensitivity and Statistical Power	6
2.5 Theoretical Justification of Disagreement Sampling	7
2.5.1 Selection Bias and the Sampling Indicator	7
2.5.2 Change of Measure and the Horvitz–Thompson Estimator	7
2.5.3 Bias–Power Trade-off in Drift Detection	8
3 Drift Monitoring and Model Selection Methodology	9
3.1 Formalisation of Temporal Evaluation	9
3.2 Statistical Framework for Drift Detection	9
3.2.1 Standard Error on the Informative Subset	9
3.2.2 The Alerting Rule	9
3.3 Accuracy–Cost Trade-off Modeling	9
3.4 Decision Logic for Model Switching	10
3.5 Statistical Drift Detection	10
3.6 Cost-Aware Model Selection	10
4 Prompt Optimisation as an Adaptive Mechanism	11
4.1 Integrating Optimisation into the Monitoring Loop	11
4.2 Pareto-Based Prompt Search (GEPA)	11
5 System Architecture and Experiments	12
5.1 Overview of the Evaluation Pipeline	12
5.2 Dataset Storage and Trace Logging	12
5.3 Batch Evaluation Process	12
5.4 Interaction with LLM APIs	13
5.5 Metrics Collection and Monitoring Dashboard	13

Contents	II
6 Conclusion	15
Bibliography	16
Statutory Declaration in Lieu of an Oath	17

1 Introduction

1.1 Context and Motivation

Industrial classification pipelines increasingly rely on Large Language Model (LLM) providers that expose their systems through application programming interfaces. In such settings, the deployed model can be viewed as a non-stationary black-box function $f_t(x)$ whose internal parameters and update schedule are controlled by the provider rather than by the user. As a result, the behaviour of the same API-accessible model may change over time even when the downstream task, the prompt, and the evaluation procedure remain fixed [1–3].

The practical motivation for this thesis arises from the author’s two-year work experience at **Gocomo GmbH**. In the company’s social-media analytics workflow, LLMs are used in batch mode to determine whether a post is relevant to a specified campaign, brand, event, or other marketing entity. This is naturally formulated as a binary classification task. Because the volume of incoming content is large, manual verification is not scalable, and automated classification becomes a necessary operational component.

However, the use of external LLM APIs creates a reliability problem. A model that performs well during an initial benchmark may later degrade due to silent provider-side updates. In a business-critical pipeline, such degradation must be detected early. At the same time, repeated full-scale benchmarking on a large labelled dataset is costly. This creates a need for a monitoring methodology that is statistically informative, operationally feasible, and economically justified.

1.2 Problem Statement

This thesis addresses the problem of how to evaluate and monitor API-based LLM classifiers in a production environment where model behaviour may change over time. The central challenge is that the user does not control the underlying model updates, yet must still make reliable deployment decisions based on observed task performance.

More specifically, the problem consists of four interconnected parts:

1. The social-media relevance detection task must be formalised in a way that permits consistent evaluation across models, prompts, and time points.
2. Because repeated evaluation on the full reference benchmark is economically expensive, a smaller monitoring subset must be constructed without losing the ability to detect practically relevant performance changes.
3. A statistical rule is required to distinguish variability in performance from genuine model drift.
4. Once multiple candidate models are monitored, deployment decisions must account not only for predictive performance but also for inference cost.

In addition, if a deployed model exhibits degraded performance, it is natural to ask whether this degradation can be mitigated without immediately replacing the model. This motivates the investigation of prompt optimisation as an adaptive mechanism within the monitoring framework.

Accordingly, the thesis does not study model drift merely as an observational phenomenon. Its objective is to develop a practically usable framework for benchmark design, drift-sensitive monitoring, cost-aware model selection, and post-drift adaptation for binary classification with black-box LLM APIs.

1.3 Research Questions

The problem stated above is investigated through the following research questions:

- RQ1:** How can the campaign-relevance detection task be formally represented as a binary classification problem for non-stationary black-box LLM APIs?
- RQ2:** How can a reduced monitoring subset be derived from a larger reference benchmark so that routine evaluation remains substantially cheaper while preserving sensitivity to model drift?
- RQ3:** Which statistical criteria and alerting rules are appropriate for detecting practically significant performance degradation over time?
- RQ4:** How should candidate models be compared and selected when deployment decisions depend on both predictive quality and inference cost?
- RQ5:** To what extent can prompt optimisation improve or recover task performance after drift has been detected, and how can such optimisation be integrated into the monitoring loop?

2 Data and Evaluation Setup

2.1 Formal Problem Definition

2.1.1 Example LLM Request Payload

During evaluation, each social-media post is converted into a structured request sent to the LLM provider API. The request consists of a system prompt describing the classification task together with user inputs containing the extracted content of the post (text, metadata, and video frames).

A simplified example payload is shown below.

NB!: MAKE A VISUALISATION IN FIGMA OF THE PAYLOAD WITH A IMAGE → STRUCTURED OUTPUT, it is literally a map function

```
[
  {
    "role": "system",
    "content": "Classify whether the post is relevant to the campaign 'Douglas_Beauty Day 2025'. Return TRUE or FALSE and explain the reasoning."
  },
  {
    "role": "user",
    "content": "Transcription: ..."
  },
  {
    "role": "user",
    "content": "Hashtags: []"
  },
  {
    "role": "user",
    "content": [
      {
        "type": "input_text",
        "text": "Video frames extracted from the post"
      },
      {
        "type": "input_image",
        "image_url": "https://.../frame1.jpg"
      },
      {
        "type": "input_image",
        "image_url": "https://.../frame2.jpg"
      }
    ]
  }
]
```


The system prompt defines the classification criteria and the list of campaign brands. The user messages contain the multimodal content of the post, including captions, transcriptions, hashtags, and selected video frames. The model output is mapped to the binary label space $\mathcal{Y} = \{0, 1\}$.

To evaluate model drift rigorously, we must first define the classification task within a probabilistic framework.

2.1.2 The Probability Space

Let $\mathcal{X}$ denote the input space of social-media posts (textual content and associated metadata) and let $\mathcal{Y} = \{0, 1\}$ be the label space, where 1 indicates relevance to a campaign and 0 indicates irrelevance. We assume that pairs (x, y) are drawn from a joint distribution P_t on $\mathcal{X} \times \mathcal{Y}$. The subscript t signifies that this distribution or the underlying labeling function may evolve over time.

2.1.3 LLM as a Parameterized Conditional Distribution

In the context of LLM-as-a-Service, the model is a "black box." We define the classifier as a function f :

$$f(x; \pi, \theta_t) \rightarrow \hat{y} \in \{0, 1\} \quad (2.1)$$

where:

- $\pi \in \mathcal{P}$ represents the **prompt**, a discrete string chosen from the space of all possible instructions $\mathcal{P}$.
- θ_t represents the **frozen model weights** provided by the API at time t . We do not have access to θ_t , but we observe its outputs.

The model's output can be viewed as an approximation of the conditional probability $P(Y|X, \pi; \theta_t)$.

2.1.4 Loss Function and Risk

To quantify performance, we define a loss function $\mathcal{L} : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}$. For this binary classification task, we utilize the zero-one loss:

$$\mathcal{L}(\hat{y}, y) = \mathbb{I}(\hat{y} \neq y) = \begin{cases} 1 & \text{if } \hat{y} \neq y \\ 0 & \text{if } \hat{y} = y \end{cases} \quad (2.2)$$

The **Expected Risk** (or generalization error) at time t for a fixed prompt π is:

$$R(f_{\pi, \theta_t}) = \mathbb{E}_{(x, y) \sim P_t} [\mathcal{L}(f(x; \pi, \theta_t), y)] \quad (2.3)$$

Since P_t is unknown, we estimate the risk using the **Empirical Risk** $\hat{R}$ over a finite dataset $\mathcal{D}$ of size n :

$$\hat{R}(f_{\pi, \theta_t}; \mathcal{D}) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}(f(x_i; \pi, \theta_t), y_i) \quad (2.4)$$

Model drift is therefore defined as a statistically significant increase in $R(f_{\pi, \theta_t})$ over time t for a fixed prompt π . The expected risk $R(f_{\pi, \theta_t})$ represents the true average loss of the classifier under the unknown data-generating distribution P_t . In the present binary classification setting with zero-one loss, this quantity is equal to the probability of misclassification. Since the underlying distribution P_t is not directly observable, the expected risk cannot be computed exactly in practice.

Instead, performance is estimated through the empirical risk $\hat{R}(f_{\pi, \theta_t}; \mathcal{D})$, which is the average loss over a finite labelled dataset $\mathcal{D}$. Under zero-one loss, the empirical risk is simply the fraction of incorrectly classified examples in the dataset. Thus, empirical risk serves as a sample-based approximation of expected risk and forms the basis for all practical evaluation and monitoring procedures in this thesis.

2.2 The 1200-Example Reference Benchmark

We formalize the task as a mapping $g : \mathcal{X} \rightarrow \{0, 1\}$. The reference benchmark consists of $N = 1200$ manually labelled examples, establishing the ground truth for initial model selection. However, evaluating all candidate models against N examples weekly is economically prohibitive.

2.2.1 Sample Size Determination via Annotation Noise Floor

The exact size of the monitoring subset, $n = 300$, is not arbitrary but is derived from the inherent noise floor of the human annotation process. In complex subjective tasks such as social-media campaign relevance, human inter-annotator disagreement establishes an irreducible error rate, generally estimated at $\epsilon \approx 0.05$ (5%). **NB!: show an example of a disagreement between two human annotators.** Consequently, any model performance fluctuation smaller than ϵ is statistically indistinguishable from background label noise. The monitoring system must therefore be calibrated to achieve a Minimum Detectable Effect (MDE) of $\Delta = 0.05$. For an observed drop to trigger a valid alert, the margin of error of our empirical accuracy estimate $\hat{A}_n$ must be strictly bounded by this noise floor.

Let $Z_{1-\alpha}$ be the critical value for a one-sided hypothesis test at significance level $\alpha = 0.05$, where $Z_{0.95} \approx 1.645$. To ensure that the margin of error does not exceed our threshold Δ , the required sample size n must satisfy:

$$Z_{1-\alpha} \sqrt{\frac{A(1-A)}{n}} \leq \Delta \quad (2.5)$$

Because $\mathcal{D}_n$ is explicitly constructed from the region of maximal model disagreement, the empirical accuracy is pushed toward the maximum-entropy state, $A \approx 0.5$. This represents the absolute worst-case variance scenario for a Bernoulli process, where $A(1-A) = 0.25$. Substituting these values yields:

$$1.645 \sqrt{\frac{0.25}{n}} \leq 0.05 \quad (2.6)$$

$$\frac{0.8225}{\sqrt{n}} \leq 0.05 \implies \sqrt{n} \geq 16.45 \quad (2.7)$$

$$n \geq 270.6 \quad (2.8)$$

To guarantee sufficient statistical power and to provide a safety margin for occasional API routing failures or dropped traces during batch processing, the theoretically required sample size of $n \geq 271$ is rounded up to $n = 300$. This demonstrates that a 300-sample subset is the theoretical minimum required to reliably detect a 5% model drift against the back-drop of human annotation noise, fully justifying the sample size under worst-case variance assumptions.

2.3 The 300-Example Monitoring Subset

To reduce operational costs, we construct a monitoring subset of $n = 300$ examples. Crucially, this is not a naive uniform random sample. It is an *informative* subset heavily weighted towards instances where models historically disagreed. Because it over-indexes on edge cases, it acts as a stress test. While it may not provide an unbiased estimate of global accuracy, it is highly sensitive to model drift.

2.4 Statistical Justification of the Monitoring Subset

The reduction from the reference benchmark ($N = 1200$) to the monitoring subset ($n = 300$) is justified by a shift from *population estimation* to *change detection*.

2.4.1 Informative Sampling via Model Disagreement

The monitoring subset was constructed by selecting instances where at least one model in the initial benchmark exhibited disagreement. Let y_i be the ground truth and $\hat{y}_{i,m}$ be the prediction of model $m \in \mathcal{M}$. The subset $\mathcal{D}_{300}$ is defined as:

$$\mathcal{D}_{300} \subset \{x_i \in \mathcal{D}_{1200} \mid \exists m_j, m_k \in \mathcal{M} : \hat{y}_{i,m_j} \neq \hat{y}_{i,m_k}\} \quad (2.9)$$

This approach is grounded in the principles of *Active Learning*, specifically *Uncertainty Sampling*. By focusing on the disagreement region, we sample from the portion of the manifold where the models' posterior probabilities $P(Y|X)$ are closest to the decision threshold.

2.4.2 Sensitivity and Statistical Power

From a hypothesis testing perspective, the goal of weekly monitoring is to reject the null hypothesis $H_0 : \Delta A = 0$ (no drift). The statistical power of a test—the probability of detecting drift when it exists—is higher on instances with high prediction variance.

In the full dataset ($N = 1200$), a large portion of instances are "easy" cases with $P(Y = 1|X) \approx 1$ or 0 . On these instances, the Bernoulli variance $p(1 - p)$ is minimal, making them "robust" to small model drifts. In contrast, the disagreement subset ($n = 300$) centers the evaluation on the "hard" cases where $p \approx 0.5$.

While $\hat{A}_{300}$ is a biased estimator of the global population accuracy, it serves as a **proxy metric** with higher sensitivity. A performance drop of 5% on the hard 300-sample subset is a leading indicator of degradation that would require a much larger (and more expensive) sample to detect on the full 1200-sample distribution. Thus, $n = 300$ provides an optimal balance between inference cost and the statistical power to detect model updates.

2.5 Theoretical Justification of Disagreement Sampling

The construction of the monitoring subset $\mathcal{D}_{300}$ via model disagreement represents a deliberate departure from simple random sampling (SRS). In this section, we formalise this approach through the lenses of *importance sampling* and *active learning*.

2.5.1 Selection Bias and the Sampling Indicator

Let $S_i \in \{0, 1\}$ be a Bernoulli random variable indicating whether the i -th instance from the reference benchmark $\mathcal{D}_N$ is included in the monitoring subset $\mathcal{D}_n$. Under SRS, $P(S_i = 1) = n/N$. In our framework, the inclusion probability is conditioned on the disagreement of a model ensemble $\mathcal{M}$:

$$P(S_i = 1 \mid x_i, \mathcal{M}) = q_i, \quad (2.10)$$

where q_i is high if the models in $\mathcal{M}$ yield conflicting predictions $\hat{y}_{i,m}$.

Consequently, the empirical accuracy on the subset, $\hat{A}_n$, is a **biased estimator** of the population accuracy A_N :

$$\mathbb{E}[\hat{A}_n] \neq A_N. \quad (2.11)$$

This bias arises because $\mathcal{D}_n$ intentionally over-represents "hard" cases near the decision boundary, for which the expected accuracy is lower than the population average.

2.5.2 Change of Measure and the Horvitz–Thompson Estimator

To reconcile the subset performance with the population mean, we can apply a **change of measure**. From the theory of *importance sampling*, an unbiased estimate of A_N can be recovered by weighting each observation by the inverse of its inclusion probability.

Let $q_i = P(S_i = 1 \mid x_i, \mathcal{M})$ denote the (possibly model-dependent) inclusion probability. The Horvitz–Thompson estimator of the population accuracy is

$$\hat{A}_{\text{HT}} = \frac{1}{N} \sum_{i \in \mathcal{D}_n} \frac{\mathbb{I}(\hat{y}_i = y_i)}{q_i}, \quad (2.12)$$

where the factor $1/q_i$ serves as an **importance weight**. In principle, this restores unbiasedness with respect to the original reference distribution. In practice, our production pipeline treats $\hat{A}_n$ primarily as a *monitoring signal*, but the existence of $\hat{A}_{\text{HT}}$ clarifies the statistical relationship between the curated subset and the underlying data-generating distribution.

2.5.3 Bias–Power Trade-off in Drift Detection

The motivation for accepting selection bias lies in a deliberate **bias–power trade-off**. For drift detection, the primary objective is not to obtain a perfectly calibrated estimate of global accuracy, but to maximise the *power* of a test for changes over time.

By concentrating on the high-entropy region around the decision boundary, we sample from those $x \in \mathcal{X}$ where the classifier is intrinsically uncertain and where small perturbations of the underlying model parameters θ_t are most likely to flip predictions. Intuitively, these are the points where the gradient of the loss with respect to θ_t is largest; small, otherwise imperceptible shifts in θ_t that would be masked by “easy” examples in $\mathcal{D}_N$ are amplified in $\mathcal{D}_n$.

Formally, we trade interpretability of $\hat{A}_n$ as a population accuracy for increased sensitivity of the drift signal:

1. **Bias Increase:** $\hat{A}_n$ systematically underestimates A_N because it is evaluated on a stress-test subset.
2. **Power Increase:** For a fixed evaluation budget n , the test statistic based on $\mathcal{D}_n$ has higher power to detect changes in accuracy than one based on an SRS of the same size, since it concentrates mass on inputs with high prediction variance.

In this sense, $\mathcal{D}_n$ is designed as an optimised *diagnostic instrument* for temporal instability, rather than as a representative sample of day-to-day traffic.

3 Drift Monitoring and Model Selection Methodology

3.1 Formalisation of Temporal Evaluation

Let $t \in \{1, \dots, T\}$ represent weekly evaluation intervals. For each model m , we track the empirical accuracy $\hat{A}_t$ on the monitoring subset $\mathcal{D}_{300}$. Because $\mathcal{D}_{300}$ consists of high-disagreement cases, $\hat{A}_t$ will be lower than the accuracy on the full benchmark, but it serves as our primary monitoring signal S_t .

3.2 Statistical Framework for Drift Detection

To distinguish between random performance fluctuation and genuine model drift, we utilize a threshold-based alerting rule.

3.2.1 Standard Error on the Informative Subset

Given that $n = 300$, the standard error of our accuracy estimate $\hat{A}_t$ is:

$$SE(\hat{A}_t) = \sqrt{\frac{\hat{A}_t(1 - \hat{A}_t)}{n}} \quad (3.1)$$

For a model with an expected accuracy of 0.70 on the hard subset, $SE \approx 0.026$ (2.6%).

3.2.2 The Alerting Rule

We define a performance drop as statistically significant if the difference between the current accuracy and the baseline accuracy $\hat{A}_0$ exceeds a confidence interval. However, for production stability, we use a **Practical Drift Threshold** τ_{drift} . An alert is triggered if:

$$\hat{A}_t < \hat{A}_0 - k \cdot SE(\hat{A}_0) - \tau_{\text{drift}} \quad (3.2)$$

where k is a scaling factor (typically $k = 2$ for a $\approx 95\%$ confidence interval) and τ_{drift} is the minimum degradation the business is willing to tolerate before intervening.

3.3 Accuracy–Cost Trade-off Modeling

Model selection is formulated as a multi-objective optimization problem. We define the **Operational Value Score** $V_t(m)$:

$$V_t(m) = \frac{\hat{A}_t(m)}{C(m)^\alpha} \quad (3.3)$$

where $C(m)$ is the cost per 1,000 requests and α is a sensitivity parameter.

- If $\alpha = 0$, we select purely based on accuracy.
- If $\alpha = 1$, we select based on "accuracy per dollar."

In our workflow, we set α based on the current budget constraints of the Gocomo pipeline.

3.4 Decision Logic for Model Switching

The system follows a three-step decision tree each week:

1. **Monitor:** Calculate V_t for all candidate models.
2. **Detect:** Check if the currently deployed model m_{curr} has triggered a drift alert.
3. **Act:** If drift is detected, calculate if another model m_{alt} provides a significantly higher V_t . If no model satisfies the threshold, trigger the **Prompt Optimisation** loop (Chapter 4).

3.5 Statistical Drift Detection

Let $\hat{A}_t$ be the empirical accuracy of a model on the monitoring subset at week t . We define the temporal performance delta as $D_t = \hat{A}_t - \hat{A}_{t-1}$. An alert is triggered if the degradation exceeds a practical operational threshold τ_{drift} :

$$D_t < -\tau_{\text{drift}} \quad (3.4)$$

This threshold must account for the natural variance of the 300-sample subset to avoid false-positive alerts.

3.6 Cost-Aware Model Selection

Predictive accuracy cannot be isolated from inference cost in a production environment. Let $C_t(m)$ be the inference cost and $A_t(m)$ be the accuracy for model m . We define a deployment utility function J_t that penalizes high costs:

$$J_t(m) = A_t(m) - \lambda C_t(m) \quad (3.5)$$

where $\lambda > 0$ represents the business's willingness to pay for marginal accuracy improvements. The system dynamically selects the model $m^* = \arg \max_m J_t(m)$ each week.

4 Prompt Optimisation as an Adaptive Mechanism

4.1 Integrating Optimisation into the Monitoring Loop

When a deployed model triggers a drift alert ($D_t < -\tau_{\text{drift}}$), switching to a more expensive model is not the only recourse. We can attempt to recover performance via prompt optimization.

4.2 Pareto-Based Prompt Search (GEPA)

We utilize a reflective evolutionary method (GEPA) [4] for prompt adaptation. Because prompt space is discrete and non-convex, greedy selection often fails. Instead, we maintain a candidate pool of prompts and evaluate their per-instance score vectors $\mathbf{s}(\Pi) = (s_1(\Pi), \dots, s_n(\Pi))$.

We retain prompts based on Pareto dominance to preserve search diversity. A prompt $\Pi^{(a)}$ dominates $\Pi^{(b)}$ if it performs at least as well on all instances and strictly better on at least one. By mutating prompts on the Pareto frontier using natural-language reflections from execution traces, we can systematically search for a prompt Π^* that maximizes $J_t(m)$ without altering the underlying API model.

5 System Architecture and Experiments

5.1 Overview of the Evaluation Pipeline

The experiments are implemented as an automated batch evaluation pipeline that connects a versioned Langfuse dataset, multiple LLM providers (OpenAI, Gemini, Claude), and a monitoring/analysis layer. Figure ?? shows the high-level flow from labeled examples to batch jobs, responses, and metrics.

Throughout this chapter, the *primary* metrics of interest are:

- **Classification accuracy** on the monitoring subset
- **Inference cost** per model (measured via token usage and provider pricing)

The core idea is that every evaluation run is:

1. Defined as a subset of the campaign relevance dataset,
2. Executed as provider-native batch jobs, and
3. Ingested back into Langfuse as traces with structured outputs and scores, so that drift can be analysed across time and models.

5.2 Dataset Storage and Trace Logging

All labels and input features for the campaign relevance task are stored in Langfuse as a dedicated dataset. Each row represents a single influencer post with a stable `custom_id`, campaign brief, caption, optional image references, and a boolean `campaign_relevant` label. During evaluation, each model call is recorded as a Langfuse trace whose metadata includes the `custom_id`, model name, provider, run identifier, latency, and token usage. This makes it possible to reconstruct historical evaluations and to filter traces by dataset version, model, or time window when analysing drift.

5.3 Batch Evaluation Process

Batch runs are orchestrated by `run_eval.py`, which:

1. Loads an evaluation subset via `load_dataset_rows`, either directly from Langfuse or from a CSV export (e.g. the disagreement subset).
2. Applies a shared prompt template to produce a normalised conversational payload (system + user messages) per example.
3. Calls provider-specific helpers (`batch_openai.py`, `batch_gemini.py`, `batch_claude.py`) that write JSONL request files and submit batch jobs.
4. Writes a manifest data/`batch_ids.json` listing all submitted batch IDs, models, and providers.

GitHub Actions workflows schedule and execute this pipeline: they trigger batch submission, poll job status until completion, and then run the ingestion scripts (`poll_and_ingest.py`, `ingest.py`) that attach responses to the correct traces in Langfuse.

5.4 Interaction with LLM APIs

All models are queried via provider-native batch APIs, but the logical payload is always the same: a schema-constrained campaign relevance call built from one labeled example. Concretely:

- **OpenAI** (`batch_openai.py`): writes JSONL lines of the form `{custom_id, method="POST", url="/v1/responses", body}` and submits the file to the OpenAI Batch API with `endpoint="/v1/re"`
- **Gemini** (`batch_gemini.py`): writes JSONL lines with a request object (contents, optional `systemInstruction`, `generationConfig`) and uploads the file to the Google GenAI batch API via `client.batches.create(model=..., src=...)`.
- **Claude** (`batch_claude.py`): builds an in-memory list of `{custom_id, params}` records and passes it directly to the Anthropic Messages Batches API via `client.messages.batches.create`; the streamed batch results are later normalised into JSONL by `download_claude_batch_results`.

In all three cases, the underlying request encodes:

- a `custom_id` linking back to the dataset row,
- a conversational payload (system instructions + user content with campaign brief and post),
- and a JSON schema (`CampaignRelevanceMinimal`) that forces the response into a `{campaign_relevant, relevancy_reasoning}` object.

Listing ?? and Listing ?? give a concrete example of the batch line and output schema used in the experiments.

5.5 Metrics Collection and Monitoring Dashboard

The monitoring stack exposes exactly two core metrics: **classification accuracy** on the monitoring subset and **inference cost** per model.

Ingestion is handled by the script `ingest.py`, which processes each batch output file as follows:

1. Parse each JSONL line and normalise the provider response (OpenAI, Gemini, Claude) into a common structure.
2. Extract the boolean `campaign_relevant` prediction from the model output.
3. Look up the ground-truth label for the corresponding `custom_id` from Langfuse or a CSV export.
4. Derive token usage (input, output, total) from the provider-specific usage fields.
5. Convert token usage into a dollar cost using the per-million-token prices in `config/models.yaml`.
6. Write a Langfuse trace/span with:
 - `metadata`: `model`, `provider`, `custom_id`, optional `run_id`;
 - the parsed input and raw provider response;
 - three scores:
 - `accuracy` $\in \{0, 1\}$ (per-trace correctness),
 - `error_type` $\in \{\text{true_positive}, \text{false_negative}, \text{true_negative}, \text{false_positive}\}$,

- `cost_usd` (total dollar cost for this trace);
- detailed `usage_details` and `cost_details` for debugging and audits.

On top of this logging layer, a small Streamlit dashboard queries Langfuse and aggregates scores over the 300-example monitoring subset. The dashboard provides:

- per-model accuracy curves over time (mean of the per-trace accuracy scores),
- confusion-matrix summaries derived from `error_type`,
- request volume and latency distributions per run,
- and cost views such as cost per run and estimated cost per 1,000 inferences.

Screenshots from this Streamlit dashboard are used in the experiments chapter to show concrete drift events and the associated accuracy–cost trade-offs. Together, `ingest.py` and the dashboard form the practical monitoring surface of the statistical surveillance loop described in the previous chapters.

6 Conclusion

This thesis provides a statistically grounded, cost-aware framework for the continuous evaluation of LLMs in production. By decoupling the initial reference benchmark from an informative monitoring subset, and pairing statistical drift detection with automated prompt optimization, organizations can maintain high classification performance despite the inherent instability of black-box LLM APIs.

Bibliography

- [1] L. Chen, M. Zaharia, and J. Zou, "How is ChatGPT's behavior changing over time?", *arXiv preprint arXiv:2307.09009*, 2023.
- [2] S. Tu et al., "ChatLog: Carefully evaluating the evolution of ChatGPT across time", *arXiv preprint arXiv:2304.14106*, 2023.
- [3] J. Echterhoff et al., "MUSCLE: A model update strategy for compatible LLM evolution", *arXiv preprint arXiv:2407.09435*, 2024.
- [4] Author1, Author2, Author3, et al., "Gepa: Reflective prompt evolution can outperform reinforcement learning", *arXiv preprint arXiv:2507.19457*, 2025.

Statutory Declaration in Lieu of an Oath

I – Georgii Burdin – do hereby declare in lieu of an oath that I have composed the presented work independently on my own and without any other resources than the ones given.

All thoughts taken directly or indirectly from external sources are correctly acknowledged.

This work has neither been previously submitted to another authority nor has it been published yet.

Mittweida, 12. March 2026

Location, Date

Georgii Burdin